```
import tensorflow as tf
import numpy as np
import pandas as pd

print("TensorFlow:", tf.__version__)
print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
```

```
TensorFlow: 2.20.0
NumPy: 2.0.2
Pandas: 2.2.2
```

```
!pip install -q transformers datasets nltk scikit-learn
```

```
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

import re
import string
import nltk

from datasets import load_dataset
```

```
nltk.download("punkt")
nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("omw-1.4")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
True
```

```
from datasets import load_dataset

go_dataset = load_dataset(
    "google-research-datasets/go_emotions",
    "simplified"
)

print(go_dataset)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
README.md: 100%  9.40k/9.40k [00:00<00:00, 599kB/s]
simplified/train-00000-of-00001.parquet: 100%  2.77M/2.77M [00:01<00:00, 2.51MB/s]
simplified/validation-00000-of-00001.par(…): 100%  350k/350k [00:00<00:00, 566kB/s]
simplified/test-00000-of-00001.parquet: 100%  347k/347k [00:00<00:00, 651kB/s]
Generating train split: 100%  43410/43410 [00:00<00:00, 237082.57 examples/s]
Generating validation split: 100%  5426/5426 [00:00<00:00, 157384.66 examples/s]
Generating test split: 100%  5427/5427 [00:00<00:00, 169549.19 examples/s]
DatasetDict({
    train: Dataset({
        features: ['text', 'labels', 'id'],
        num_rows: 43410
    })
    validation: Dataset({
        features: ['text', 'labels', 'id'],
        num_rows: 5426
    })
    test: Dataset({
        features: ['text', 'labels', 'id'],
        num_rows: 5427
    })
})
```

```
from datasets import load_dataset

go_dataset = load_dataset(
    "google-research-datasets/go_emotions",
```

```
    "simplified"
)

print(go_dataset)
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'labels', 'id'],
        num_rows: 43410
    })
    validation: Dataset({
        features: ['text', 'labels', 'id'],
        num_rows: 5426
    })
    test: Dataset({
        features: ['text', 'labels', 'id'],
        num_rows: 5427
    })
})
```

```
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
True
```

```
from datasets import load_dataset
import pandas as pd
```

```
go_dataset = load_dataset(
    "google-research-datasets/go_emotions",
    "simplified"
)
```

```
train_df = go_dataset["train"].to_pandas()
```

```
train_df.head()
```

	text	labels	id
0	My favourite food is anything I didn't have to...	[27]	eebbqej
1	Now if he does off himself, everyone will thin...	[27]	ed00q6i
2	WHY THE FUCK IS BAYLESS ISOING	[2]	eezlygj
3	To make her feel threatened	[14]	ed7ypvh
4	Dirty Southern Wankers	[3]	ed0bdzj

```
train_df
```

	text	labels	id
0	My favourite food is anything I didn't have to...	[27]	eebbqej
1	Now if he does off himself, everyone will thin...	[27]	ed00q6i
2	WHY THE FUCK IS BAYLESS ISOING	[2]	eezlygj
3	To make her feel threatened	[14]	ed7ypvh
4	Dirty Southern Wankers	[3]	ed0bdzj
...	...	...	...
43405	Added you mate well I've just got the bow and ...	[18]	edsb738
43406	Always thought that was funny but is it a refe...	[6]	ee7fdou
43407	What are you talking about? Anything bad that ...	[3]	efgbhks
43408	More like a baptism, with sexy results!	[13]	ed1naf8
43409	Enjoy the ride!	[17]	eecwmbq

43410 rows × 3 columns

```
import pandas as pd
from datasets import load_dataset
```

```
go_dataset = load_dataset(
    "google-research-datasets/go_emotions",
    "simplified"
)
```

```
train_df.head()
```

	text	labels	id
0	My favourite food is anything I didn't have to...	[27]	eebbqej
1	Now if he does off himself, everyone will thin...	[27]	ed00q6i
2	WHY THE FUCK IS BAYLESS ISOING	[2]	eezlygj
3	To make her feel threatened	[14]	ed7ypvh
4	Dirty Southern Wankers	[3]	ed0bdzj

```
train_df = go_dataset["train"].to_pandas()
```

```
train_df.head()
```

	text	labels	id
0	My favourite food is anything I didn't have to...	[27]	eebbqej
1	Now if he does off himself, everyone will thin...	[27]	ed00q6i
2	WHY THE FUCK IS BAYLESS ISOING	[2]	eezlygj
3	To make her feel threatened	[14]	ed7ypvh
4	Dirty Southern Wankers	[3]	ed0bdzj

```
train_df.to_csv("goemotions_train.csv", index=False)

print("Dataset saved successfully!")
```

```
Dataset saved successfully!
```

```
import os

print(os.listdir())
```

```
['.config', 'goemotions_train.csv', 'sample_data']
```

```
print(train_df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 43410 entries, 0 to 43409
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   text    43410 non-null  object
 1   labels  43410 non-null  object
 2   id      43410 non-null  object
dtypes: object(3)
memory usage: 1017.6+ KB
None
```

```
import pandas as pd
import numpy as np
import re
import string
import nltk

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

```
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
True
```

```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()
```

```
def clean_text(text):

    text = text.lower()

    text = re.sub(r"http\S+|www\S+", "", text)

    text = re.sub(r"<.*?>", "", text)

    text = text.translate(str.maketrans("", "", string.punctuation))

    text = re.sub(r"\d+", "", text)

    text = text.strip()

    words = text.split()

    words = [
        lemmatizer.lemmatize(word)
        for word in words
        if word not in stop_words
    ]

    return " ".join(words)
```

```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def clean_text(text):

    # Convert to lowercase
    text = text.lower()

    # Remove URLs
    text = re.sub(r"http\S+|www\S+", "", text)

    # Remove HTML tags
    text = re.sub(r"<.*?>", "", text)

    # Remove punctuation
    text = text.translate(str.maketrans("", "", string.punctuation))

    # Remove numbers
    text = re.sub(r"\d+", "", text)

    # Remove extra spaces
    text = text.strip()

    # Tokenize
    words = text.split()

    # Remove stopwords
    words = [word for word in words if word not in stop_words]

    # Lemmatize
    words = [lemmatizer.lemmatize(word) for word in words]

    return " ".join(words)
```

```
train_df["clean_text"] = train_df["text"].apply(clean_text)
```

```
train_df[["text", "clean_text"]].head(10)
```

	text	clean_text
0	My favourite food is anything I didn't have to...	favourite food anything didnt cook
1	Now if he does off himself, everyone will thin...	everyone think he laugh screwing people instea...
2	WHY THE FUCK IS BAYLESS ISOING	fuck bayless isoing
3	To make her feel threatened	make feel threatened
4	Dirty Southern Wankers	dirty southern wanker
5	OmG pEyToN iSn'T gOoD eNoUgH tO hElP uS iN tHe...	omg peyton isnt good enough help u playoff dum...
6	Yes I heard abt the f bombs! That has to be wh...	yes heard abt f bomb thanks reply hubby anxiou...
7	We need more boards and to create a bit more s...	need board create bit space name we'll good
8	Damn youtube and outrage drama is super lucrat...	damn youtube outrage drama super lucrative reddit
9	It might be linked to the trust factor of your...	might linked trust factor friend

```
emotion_keywords = {

    "Happy": [
        "happy", "joy", "love", "great",
        "excellent", "excited", "wonderful"
    ],

    "Sad": [
        "sad", "cry", "depressed",
        "hurt", "lonely", "upset"
    ],

    "Angry": [
        "angry", "mad", "annoyed",
        "furious", "hate"
    ],

    "Fear": [
        "fear", "afraid", "scared",
        "worried", "nervous", "anxious"
    ],

    "Neutral": []
}
```

```
def extract_keywords(text):

    words = text.split()

    found_keywords = []

    for emotion, keyword_list in emotion_keywords.items():

        for word in words:

            if word in keyword_list:
                found_keywords.append(word)

    return found_keywords
```

```
train_df["keywords"] = train_df["clean_text"].apply(extract_keywords)
```

```
train_df[["clean_text", "keywords"]].head(20)
```

	clean_text	keywords
0	favourite food anything didnt cook	[]
1	everyone think he laugh screwing people instea...	[]
2	fuck bayless isoing	[]
3	make feel threatened	[]
4	dirty southern wanker	[]
5	omg peyton isnt good enough help u playoff dum...	[]
6	yes heard abt f bomb thanks reply hubby anxiou...	[]
7	need board create bit space name we'll good	[]
8	damn youtube outrage drama super lucrative reddit	[]
9	might linked trust factor friend	[]
10	demographic don't know anybody cable tv	[]
11	aww shell probably come around eventually im s...	[]
12	hello everyone im toronto well call visit pers...	[]
13	rsleeptrain might time sleep training take loo...	[]
14	name fucking problem slightly better command e...	[]
15	shit guess accidentally bought payperview boxi...	[]
16	thank friend	[]
17	fucking coward	[]
18	retardation look like	[]
19	maybe that's happened great white houston zoo	[great]

```
train_df.to_csv("processed_goemotions.csv", index=False)

print("Processed dataset saved successfully.")
```

```
Processed dataset saved successfully.
```

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding
from tensorflow.keras.layers import Bidirectional
from tensorflow.keras.layers import LSTM
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
```

```
train_df = pd.read_csv("processed_goemotions.csv")

train_df.head()
```

	text	labels	id	clean_text	keywords
0	My favourite food is anything I didn't have to...	[27]	eebbqej	favourite food anything didnt cook	[]
1	Now if he does off himself, everyone will thin...	[27]	ed00q6i	everyone think he laugh screwing people instea...	[]
2	WHY THE FUCK IS BAYLESS ISOING	[2]	eezlygj	fuck bayless isoing	[]
3	To make her feel threatened	[14]	ed7ypvh	make feel threatened	[]
4	Dirty Southern Wankers	[3]	ed0bdzj	dirty southern wanker	[]

```
print(train_df["labels"].head())
```

```
0    [27]
1    [27]
2     [2]
3    [14]
4     [3]
Name: labels, dtype: object
```

```
import ast
import pandas as pd
```

```
train_df["labels"].head(20)
```

	labels
0	[27]
1	[27]
2	[2]
3	[14]
4	[3]
5	[26]
6	[15]
7	[8 20]
8	[0]
9	[27]
10	[6]
11	[1 4]
12	[27]
13	[5]
14	[3]
15	[3 12]
16	[15]
17	[2]
18	[27]
19	[6 22]

dtype: object

```
train_df["labels"].sample(10)
```

	labels
41652	[9]
20713	[4]
2447	[0]
30496	[10]
5555	[0 21]
11773	[15]
41989	[4]
37846	[9]
4378	[27]
16052	[18 22]

dtype: object

```
import re

def extract_first_label(label):
    numbers = re.findall(r'\d+', str(label))

    if len(numbers) == 0:
        return None

    return int(numbers[0])

train_df["label_id"] = train_df["labels"].apply(extract_first_label)
```

```
train_df[["labels", "label_id"]].head(20)
```

	labels	label_id
0	[27]	27
1	[27]	27
2	[2]	2
3	[14]	14
4	[3]	3
5	[26]	26
6	[15]	15
7	[8 20]	8
8	[0]	0
9	[27]	27
10	[6]	6
11	[1 4]	1
12	[27]	27
13	[5]	5
14	[3]	3
15	[3 12]	3
16	[15]	15
17	[2]	2
18	[27]	27
19	[6 22]	6

```
emotion_map = {
    # Happy
    0: "Happy",      # admiration
    1: "Happy",      # amusement
    4: "Happy",      # approval
    6: "Happy",      # caring
    13: "Happy",     # gratitude
    16: "Happy",     # excitement
    17: "Happy",     # joy
    18: "Happy",     # love
    20: "Happy",     # optimism
    21: "Happy",     # pride
    23: "Happy",     # relief

    # Angry
    2: "Angry",      # anger
    3: "Angry",      # annoyance
    11: "Angry",     # disgust

    # Sad
    9: "Sad",        # disappointment
    12: "Sad",       # embarrassment
    14: "Sad",       # grief
    24: "Sad",       # remorse
    25: "Sad",       # sadness

    # Fear
    10: "Fear",      # fear
    19: "Fear",      # nervousness

    # Neutral
    5: "Neutral",    # confusion
    7: "Neutral",    # curiosity
    8: "Neutral",    # desire
    15: "Neutral",   # realization
    22: "Neutral",   # surprise
    26: "Neutral",   # neutral
    27: "Neutral"
}
```

```
train_df["emotion"] = train_df["label_id"].map(emotion_map)
```

```
print(train_df["emotion"].isnull().sum())
```

```
0
```

```
print(train_df["emotion"].isnull().sum())
```

```
0
```

```
train_df["emotion"].value_counts()
```

emotion	count
Neutral	19649
Happy	14676
Angry	4265
Sad	3064
Fear	1756

dtype: int64

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

train_df["target"] = encoder.fit_transform(train_df["emotion"])

print(encoder.classes_)
```

```
['Angry' 'Fear' 'Happy' 'Neutral' 'Sad']
```

```
train_df["clean_text"] = train_df["clean_text"].astype(str)
```

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

MAX_WORDS = 10000
MAX_LENGTH = 100

tokenizer = Tokenizer(num_words=MAX_WORDS)

tokenizer.fit_on_texts(train_df["clean_text"])

X = tokenizer.texts_to_sequences(train_df["clean_text"])

X = pad_sequences(X, maxlen=MAX_LENGTH)

y = train_df["target"]
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
```

```
(34728, 100)
(8682, 100)
```

```
print(train_df["emotion"].isnull().sum())
```

```
0
```

```
print(X_train.shape)
print(X_test.shape)
```

```
(34728, 100)
(8682, 100)
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM
```

```
from tensorflow.keras.layers import Dense, Dropout
```

```
VOCAB_SIZE = 10000
EMBEDDING_DIM = 128
MAX_LENGTH = 100

model = Sequential([
    Embedding(
        input_dim=VOCAB_SIZE,
        output_dim=EMBEDDING_DIM,
        input_length=MAX_LENGTH
    ),

    Bidirectional(
        LSTM(64)
    ),

    Dropout(0.5),

    Dense(32, activation='relu'),

    Dense(5, activation='softmax')
])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is depr
  warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
bidirectional (Bidirectional)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=5,
    batch_size=64,
    verbose=1
)
```

```
Epoch 1/5
435/435 ━━━━━━━━━━━━━━━━━━━━ 119s 255ms/step - accuracy: 0.5533 - loss: 1.1503 - val_accuracy: 0.6504 - val_loss: 0.9726
Epoch 2/5
435/435 ━━━━━━━━━━━━━━━━━━━━ 113s 261ms/step - accuracy: 0.6835 - loss: 0.8758 - val_accuracy: 0.6434 - val_loss: 0.9520
Epoch 3/5
435/435 ━━━━━━━━━━━━━━━━━━━━ 111s 254ms/step - accuracy: 0.7273 - loss: 0.7424 - val_accuracy: 0.6227 - val_loss: 1.0045
Epoch 4/5
435/435 ━━━━━━━━━━━━━━━━━━━━ 108s 248ms/step - accuracy: 0.7631 - loss: 0.6434 - val_accuracy: 0.6142 - val_loss: 1.1171
Epoch 5/5
435/435 ━━━━━━━━━━━━━━━━━━━━ 142s 249ms/step - accuracy: 0.7969 - loss: 0.5477 - val_accuracy: 0.5940 - val_loss: 1.2701
```

Evaluate Model Performance

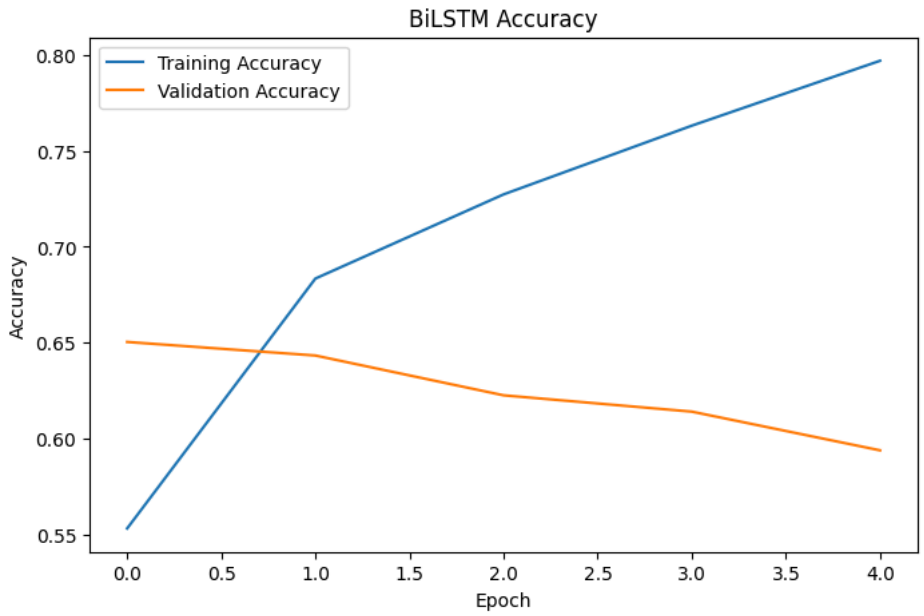
Now that the model has been trained, we need to evaluate its performance on the unseen test set to understand how well it generalizes to new data.

```
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print(f"Test Loss: {loss:.4f}")
print(f"Test Accuracy: {accuracy:.4f}")
```

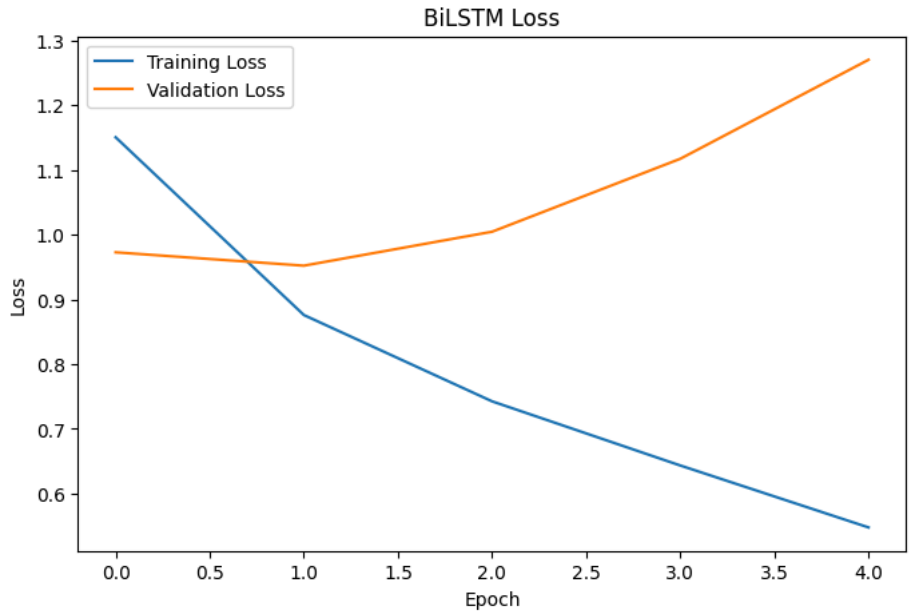
```
loss, accuracy = model.evaluate(
    X_test,
    y_test
)

print("Test Accuracy:", accuracy)
print("Test Loss:", loss)
```

```
272/272 ━━━━━━━━━━━━━━━━━━━━ 9s 34ms/step - accuracy: 0.6009 - loss: 1.2233
Test Accuracy: 0.6008983850479126
Test Loss: 1.2233089208602905
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("BiLSTM Accuracy")

plt.legend()

plt.show()
```




```
plt.figure(figsize=(8,5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("BiLSTM Loss")

plt.legend()

plt.show()
```




```
model.save("bilstm_model.keras")

print("BiLSTM model saved successfully!")
```

```
BiLSTM model saved successfully!
```

```
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

class_weights = compute_class_weight(
    class_weight="balanced",
    classes=np.unique(y_train),
    y=y_train
)

class_weights = dict(enumerate(class_weights))
print(class_weights)
```

```
{0: np.float64(2.033255269320843), 1: np.float64(4.884388185654008), 2: np.float64(0.5930834258389548), 3: np.float64(0.4417
```

```
history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=5,
    batch_size=64,
    class_weight=class_weights,
    verbose=1
)
```

```
Epoch 1/5
435/435 ━━━━━━━━━━━━━━━━━━━━ 0s 241ms/step - accuracy: 0.7956 - loss: 0.5789
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_801/3347120760.py in <cell line: 0>()
----> 1 history = model.fit(
      2     X_train,
      3     y_train,
      4     validation_split=0.2,
      5     epochs=5,

 1 frames
/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/trainer.py in fit(self, x, y, batch_size, epochs,
verbose, callbacks, validation_split, validation_data, shuffle, class_weight, sample_weight, initial_epoch,
steps_per_epoch, validation_steps, validation_batch_size, validation_freq)
    432                 )
    433                 val_logs = {
--> 434                     f"val_{name}": val for name, val in val_logs.items()
    435                 }
    436                 epoch_logs.update(val_logs)

KeyboardInterrupt:
```

```
import tensorflow as tf
```

```
print("TensorFlow:", tf.__version__)
print("GPU Available:", tf.config.list_physical_devices('GPU'))
```

```
TensorFlow: 2.20.0
GPU Available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
!pip install -q transformers datasets
```

```
import transformers
import tensorflow as tf

print("Transformers:", transformers.__version__)
print("TensorFlow:", tf.__version__)
```

```
Transformers: 5.12.1
TensorFlow: 2.20.0
```

```
import transformers

print(hasattr(transformers, "TFBertForSequenceClassification"))
```

```
False
```

```
!pip install -q transformers datasets accelerate evaluate
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 3.1 MB/s eta 0:00:00
```

```
import pandas as pd
import numpy as np
import torch

from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

```
print("PyTorch Version:", torch.__version__)
print("GPU Available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
```

```
PyTorch Version: 2.11.0+cu128
GPU Available: True
Tesla T4
```

```
import os

print(os.listdir())
```

```
['.config', 'sample_data']
```

```
from datasets import load_dataset

go_dataset = load_dataset(
    "google-research-datasets/go_emotions",
    "simplified"
)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
README.md: 100%  9.40k/9.40k [00:00<00:00, 734kB/s]
simplified/train-00000-of-00001.parquet: 100%  2.77M/2.77M [00:02<00:00, 1.37MB/s]
simplified/validation-00000-of-00001.par(…): 100%  350k/350k [00:00<00:00, 382kB/s]
simplified/test-00000-of-00001.parquet: 100%  347k/347k [00:00<00:00, 417kB/s]
Generating train split: 100%  43410/43410 [00:00<00:00, 496867.84 examples/s]
Generating validation split: 100%  5426/5426 [00:00<00:00, 229323.50 examples/s]
Generating test split: 100%  5427/5427 [00:00<00:00, 158209.07 examples/s]
```

```
import pandas as pd

train_df = go_dataset["train"].to_pandas()

train_df.head()
```

	text	labels	id
0	My favourite food is anything I didn't have to...	[27]	eebbqej
1	Now if he does off himself, everyone will thin...	[27]	ed00q6i
2	WHY THE FUCK IS BAYLESS ISOING	[2]	eezlygj
3	To make her feel threatened	[14]	ed7ypvh
4	Dirty Southern Wankers	[3]	ed0bdzj

```
print(train_df.columns)
```

```
Index(['text', 'labels', 'id'], dtype='object')
```

```
train_df.to_csv("goemotions_train.csv", index=False)

print("Saved Successfully")
```

```
Saved Successfully
```

```
import os

print(os.listdir())
```

```
['.config', 'goemotions_train.csv', 'sample_data']
```

```
import pandas as pd

train_df = pd.read_csv("goemotions_train.csv")

train_df.head()
```

	text	labels	id
0	My favourite food is anything I didn't have to...	[27]	eebbqej
1	Now if he does off himself, everyone will thin...	[27]	ed00q6i
2	WHY THE FUCK IS BAYLESS ISOING	[2]	eezlygj
3	To make her feel threatened	[14]	ed7ypvh
4	Dirty Southern Wankers	[3]	ed0bdzj

```
import re
import string
import nltk

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

nltk.download("stopwords")
nltk.download("wordnet")
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def clean_text(text):

    text = str(text).lower()

    # Remove URLs
    text = re.sub(r"http\S+|www\S+", "", text)

    # Remove HTML tags
    text = re.sub(r"<.*?>", "", text)

    # Remove punctuation
    text = text.translate(str.maketrans("", "", string.punctuation))

    # Remove numbers
    text = re.sub(r"\d+", "", text)

    # Remove extra spaces
    text = text.strip()

    # Tokenize
    words = text.split()

    # Remove stopwords
    words = [word for word in words if word not in stop_words]

    # Lemmatize
    words = [lemmatizer.lemmatize(word) for word in words]

    return " ".join(words)
```

```
train_df["clean_text"] = train_df["text"].apply(clean_text)
```

```
train_df[["text", "clean_text"]].head(10)
```

	text	clean_text
0	My favourite food is anything I didn't have to...	favourite food anything didnt cook
1	Now if he does off himself, everyone will thin...	everyone think he laugh screwing people instea...
2	WHY THE FUCK IS BAYLESS ISOING	fuck bayless isoing
3	To make her feel threatened	make feel threatened
4	Dirty Southern Wankers	dirty southern wanker
5	OmG pEyToN iSn'T gOoD eNoUgH tO hElP uS iN tHe...	omg peyton isnt good enough help u playoff dum...
6	Yes I heard abt the f bombs! That has to be wh...	yes heard abt f bomb thanks reply hubby anxiou...
7	We need more boards and to create a bit more s...	need board create bit space name we'll good
8	Damn youtube and outrage drama is super lucrat...	damn youtube outrage drama super lucrative reddit
9	It might be linked to the trust factor of your...	might linked trust factor friend

```
train_df.to_csv("processed_goemotions.csv", index=False)

print("Processed dataset saved successfully.")
```

```
Processed dataset saved successfully.
```

```
import os

print(os.listdir())
```

```
['.config', 'processed_goemotions.csv', 'goemotions_train.csv', 'sample_data']
```

```
import re

def extract_first_label(label):
    numbers = re.findall(r"\d+", str(label))

    if len(numbers) == 0:
        return None

    return int(numbers[0])

train_df["label_id"] = train_df["labels"].apply(extract_first_label)
```

```
train_df[["labels", "label_id"]].head(10)
```

	labels	label_id
0	[27]	27
1	[27]	27
2	[2]	2
3	[14]	14
4	[3]	3
5	[26]	26
6	[15]	15
7	[8 20]	8
8	[0]	0
9	[27]	27

```
emotion_map = {

    # Happy
    0: "Happy",
    1: "Happy",
    4: "Happy",
    6: "Happy",
    13: "Happy",
    16: "Happy",
    17: "Happy",
    18: "Happy",
    20: "Happy",
    21: "Happy",
    23: "Happy",

    # Angry
    2: "Angry",
    3: "Angry",
    11: "Angry",

    # Sad
    9: "Sad",
    12: "Sad",
    14: "Sad",
    24: "Sad",
    25: "Sad",

    # Fear
    10: "Fear",
    19: "Fear",

    # Neutral
    5: "Neutral",
    7: "Neutral",
    8: "Neutral",
    15: "Neutral",
    22: "Neutral",
    26: "Neutral",
    27: "Neutral"
}

train_df["emotion"] = train_df["label_id"].map(emotion_map)

train_df[["label_id", "emotion"]].head(10)
```

	label_id	emotion
0	27	Neutral
1	27	Neutral
2	2	Angry
3	14	Sad
4	3	Angry
5	26	Neutral
6	15	Neutral
7	8	Neutral
8	0	Happy
9	27	Neutral

```
print(train_df["emotion"].isnull().sum())
```

```
0
```

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

train_df["target"] = encoder.fit_transform(train_df["emotion"])

print(encoder.classes_)
```

```
['Angry' 'Fear' 'Happy' 'Neutral' 'Sad']
```

```
train_df.to_csv("processed_goemotions.csv", index=False)

print("Dataset updated successfully.")
```

```
Dataset updated successfully.
```

```
print(train_df.columns)
```

```
Index(['text', 'labels', 'id', 'clean_text', 'label_id', 'emotion', 'target'], dtype='object')
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    train_df["clean_text"],
    train_df["target"],
    test_size=0.2,
    random_state=42,
    stratify=train_df["target"]
)

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

```
Training Samples: 34728
Testing Samples: 8682
```

```
from datasets import Dataset

train_dataset = Dataset.from_dict({
    "text": list(X_train),
    "label": list(y_train)
})

test_dataset = Dataset.from_dict({
    "text": list(X_test),
    "label": list(y_test)
})

print(train_dataset)
```

```
Dataset({
    features: ['text', 'label'],
    num_rows: 34728
})
```

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

```
config.json: 100%  570/570 [00:00<00:00, 30.9kB/s]
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 1.54kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 2.10MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 3.37MB/s]
```

```
def tokenize(batch):
    return tokenizer(
        batch["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

train_dataset = train_dataset.map(tokenize, batched=True)
test_dataset = test_dataset.map(tokenize, batched=True)
```

```
Map: 100%  34728/34728 [00:06<00:00, 8985.77 examples/s]
Map: 100%  8682/8682 [00:01<00:00, 5279.71 examples/s]
```

```
train_dataset
```

```
Dataset({
    features: ['text', 'label', 'input_ids', 'token_type_ids', 'attention_mask'],
    num_rows: 34728
})
```

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=5
)
```

```
model.safetensors: 100%  440M/440M [00:07<00:00, 63.1MB/s]
Loading weights: 100%  199/199 [00:00<00:00, 3027.37it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
--------------------------------------------+------------+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.weight                 | UNEXPECTED |
cls.predictions.transform.dense.bias        | UNEXPECTED |
cls.seq_relationship.bias                   | UNEXPECTED |
cls.predictions.bias                        | UNEXPECTED |
cls.predictions.transform.dense.weight      | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |
classifier.bias                             | MISSING    |
classifier.weight                           | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING:  those params were newly initialized because missing from the checkpoint. Consider training on your downstream ta
```

```
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

classes = np.unique(y_train)

weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train
)

class_weights = dict(zip(classes, weights))

print(class_weights)
```

```
{np.int64(0): np.float64(2.035638921453693), np.int64(1): np.float64(4.943487544483986), np.int64(2): np.float64(0.591568009
```

```
train_dataset = train_dataset.remove_columns(["text"])
test_dataset = test_dataset.remove_columns(["text"])
```

```
train_dataset.set_format(
    type="torch",
    columns=["input_ids", "attention_mask", "label"]
)

test_dataset.set_format(
    type="torch",
    columns=["input_ids", "attention_mask", "label"]
)
```

```
import evaluate
import numpy as np

accuracy = evaluate.load("accuracy")
f1 = evaluate.load("f1")

def compute_metrics(eval_pred):

    logits, labels = eval_pred

    predictions = np.argmax(logits, axis=-1)

    acc = accuracy.compute(
        predictions=predictions,
        references=labels
    )

    f1_score = f1.compute(
        predictions=predictions,
        references=labels,
        average="weighted"
    )

    return {
        "accuracy": acc["accuracy"],
        "f1": f1_score["f1"]
    }
```

```
Downloading builder script:  4.20k/? [00:00<00:00, 133kB/s]

Downloading builder script:  6.79k/? [00:00<00:00, 385kB/s]
```

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./bert_results",

    eval_strategy="epoch",

    save_strategy="epoch",

    learning_rate=2e-5,

    per_device_train_batch_size=16,

    per_device_eval_batch_size=16,

    num_train_epochs=3,

    weight_decay=0.01,

    logging_steps=100,

    load_best_model_at_end=True,

    report_to="none"
)
```

```
from transformers import Trainer

trainer = Trainer(
    model=model,

    args=training_args,

    train_dataset=train_dataset,

    eval_dataset=test_dataset,
```

```
    compute_metrics=compute_metrics
)
```

```
!pip install -q transformers datasets accelerate
```

```
import torch
import torchvision
import transformers

print("Torch:", torch.__version__)
print("Torchvision:", torchvision.__version__)
print("Transformers:", transformers.__version__)
```

```
Torch: 2.11.0+cu128
Torchvision: 0.26.0+cu128
Transformers: 5.12.1
```

```
!pip uninstall -y transformers
!pip install -q transformers==4.52.4 accelerate datasets evaluate
```

```
Found existing installation: transformers 5.12.1
Uninstalling transformers-5.12.1:
  Successfully uninstalled transformers-5.12.1
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.5/10.5 MB 79.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 566.4/566.4 kB 44.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.1/3.1 MB 84.4 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
gradio 6.19.0 requires huggingface-hub<2.0,>=1.2.0, but you have huggingface-hub 0.36.2 which is incompatible.
```

```
import transformers
print(transformers.__version__)
```

```
5.12.1
```

```
!pip uninstall -y transformers tokenizers datasets accelerate evaluate
```

```
Found existing installation: transformers 4.52.4
Uninstalling transformers-4.52.4:
  Successfully uninstalled transformers-4.52.4
Found existing installation: tokenizers 0.21.4
Uninstalling tokenizers-0.21.4:
  Successfully uninstalled tokenizers-0.21.4
Found existing installation: datasets 4.0.0
Uninstalling datasets-4.0.0:
  Successfully uninstalled datasets-4.0.0
Found existing installation: accelerate 1.14.0
Uninstalling accelerate-1.14.0:
  Successfully uninstalled accelerate-1.14.0
Found existing installation: evaluate 0.4.6
Uninstalling evaluate-0.4.6:
  Successfully uninstalled evaluate-0.4.6
```

```
!pip install -q \
transformers==4.52.4 \
datasets==3.6.0 \
accelerate==1.7.0 \
evaluate==0.4.3
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 491.5/491.5 kB 19.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 362.1/362.1 kB 33.6 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.0/84.0 kB 8.7 MB/s eta 0:00:00
```

```
import transformers
import datasets
import accelerate
import evaluate

print("Transformers:", transformers.__version__)
print("Datasets:", datasets.__version__)
print("Accelerate:", accelerate.__version__)
print("Evaluate:", evaluate.__version__)
```

```
Transformers: 5.12.1
Datasets: 4.0.0
Accelerate: 1.14.0
Evaluate: 0.4.6
```

```
!pip uninstall -y transformers
!pip install -q transformers==4.52.4 datasets==3.6.0 accelerate==1.7.0 evaluate==0.4.3
```

```
Found existing installation: transformers 4.52.4
Uninstalling transformers-4.52.4:
  Successfully uninstalled transformers-4.52.4
```

```
import transformers
import datasets
import accelerate
import evaluate

print(transformers.__version__)
print(datasets.__version__)
print(accelerate.__version__)
print(evaluate.__version__)
```

```
5.12.1
4.0.0
1.14.0
0.4.6
```

```
import transformers
import datasets
import accelerate
import evaluate

print("Transformers:", transformers.__version__)
print("Datasets:", datasets.__version__)
print("Accelerate:", accelerate.__version__)
print("Evaluate:", evaluate.__version__)
```

```
Transformers: 5.12.1
Datasets: 4.0.0
Accelerate: 1.14.0
Evaluate: 0.4.6
```

```
import torch
from torch.utils.data import Dataset, DataLoader
import pandas as pd
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import numpy as np
```

```
train_df = pd.read_csv("processed_goemotions.csv")

train_df = train_df.dropna(subset=["clean_text", "target"])

texts = train_df["clean_text"].tolist()
labels = train_df["target"].tolist()

print(len(texts), len(labels))
```

```
43318 43318
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

```
class EmotionDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=128):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts[idx])
        label = self.labels[idx]

        encoding = self.tokenizer(
            text,
            truncation=True,
            padding="max_length",
            max_length=self.max_len,
            return_tensors="pt"
        )

        return {
            "input_ids": encoding["input_ids"].squeeze(),
            "attention_mask": encoding["attention_mask"].squeeze(),
```

```
            "labels": torch.tensor(label, dtype=torch.long)
        }
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    texts,
    labels,
    test_size=0.2,
    random_state=42,
    stratify=labels
)

train_dataset = EmotionDataset(X_train, y_train, tokenizer)
test_dataset = EmotionDataset(X_test, y_test, tokenizer)
```

```
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16)
```

```
batch = next(iter(train_loader))
print(batch["input_ids"].shape)
print(batch["labels"][:5])
```

```
torch.Size([16, 128])
tensor([2, 3, 3, 3, 4])
```

```
from transformers import AutoModelForSequenceClassification

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=5
)

model.to(device)
```

```
Device: cuda
```

```
Loading weights: 100%                                                    199/199 [00:00<00:00, 3505.96it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
--------------------------------------------+------------+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.weight                | UNEXPECTED |
cls.predictions.transform.dense.bias       | UNEXPECTED |
cls.seq_relationship.bias                  | UNEXPECTED |
cls.predictions.bias                       | UNEXPECTED |
cls.predictions.transform.dense.weight     | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |
classifier.bias                            | MISSING    |
classifier.weight                          | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING:  those params were newly initialized because missing from the checkpoint. Consider training on your downstream
BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
```

```
from torch.optim import AdamW
import torch.nn as nn

optimizer = AdamW(model.parameters(), lr=2e-5)
loss_fn = nn.CrossEntropyLoss()
```

```
epochs = 3

for epoch in range(epochs):
    model.train()
    total_loss = 0

    for batch in train_loader:
        optimizer.zero_grad()

        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask
        )

        loss = loss_fn(outputs.logits, labels)
        loss.backward()
        optimizer.step()
```

```
        total_loss += loss.item()

    print(f"Epoch {epoch+1} Loss: {total_loss/len(train_loader):.4f}")
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_2212/3662440598.py in <cell line: 0>()
     21         optimizer.step()
     22
---> 23         total_loss += loss.item()
     24
     25     print(f"Epoch {epoch+1} Loss: {total_loss/len(train_loader):.4f}")

KeyboardInterrupt:
```

```
import torch

print("CUDA available:", torch.cuda.is_available())
print("Device:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU")
```

```
CUDA available: True
Device: Tesla T4
```

```
print(next(model.parameters()).device)
```

```
cuda:0
```

```
train_loader = DataLoader(
    train_dataset,
    batch_size=16,
    shuffle=True,
    num_workers=2,
    pin_memory=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False,
    num_workers=2,
    pin_memory=True
)
```

```
from tqdm import tqdm

model.eval()

correct = 0
total = 0

with torch.no_grad():
    for batch in tqdm(test_loader):

        input_ids = batch["input_ids"].to(device, non_blocking=True)
        attention_mask = batch["attention_mask"].to(device, non_blocking=True)
        labels = batch["labels"].to(device, non_blocking=True)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask
        )

        preds = torch.argmax(outputs.logits, dim=1)

        correct += (preds == labels).sum().item()
        total += labels.size(0)

accuracy = correct / total

print("Test Accuracy:", accuracy)
```

```
100%|██████████| 271/271 [01:01<00:00,  4.42it/s]Test Accuracy: 0.6731301939058172
```

```
label_names = encoder.classes_
print(label_names)
```

```
['Angry' 'Fear' 'Happy' 'Neutral' 'Sad']
```

```
def predict_mixed(text):

    model.eval()

    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=128
    ).to(device)

    with torch.no_grad():
        outputs = model(**inputs)
        probs = torch.softmax(outputs.logits, dim=1).cpu().numpy()[0]

    # Top prediction
    top_idx = probs.argmax()
    primary = label_names[top_idx]
    primary_prob = probs[top_idx]

    # Find secondary emotions ≥ 15%
    secondary = []
    for i, p in enumerate(probs):
        if i != top_idx and p >= 0.15:
            secondary.append((label_names[i], round(float(p)*100, 2)))

    return {
        "text": text,
        "primary_emotion": primary,
        "primary_confidence": round(float(primary_prob)*100, 2),
        "secondary_emotions": secondary
    }
```

```
result = predict_mixed("I am happy but also a little nervous about results")

print(result)
```

```
{'text': 'I am happy but also a little nervous about results', 'primary_emotion': 'Happy', 'primary_confidence': 55.92, 'sec
```

```
def unified_predict(text):

    result = predict_mixed(text)

    output = {
```